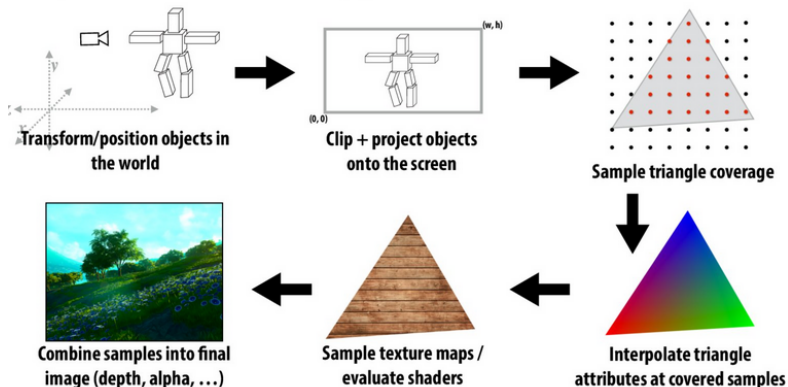


Računarska grafika

Vidljivost

Vesna Marinković

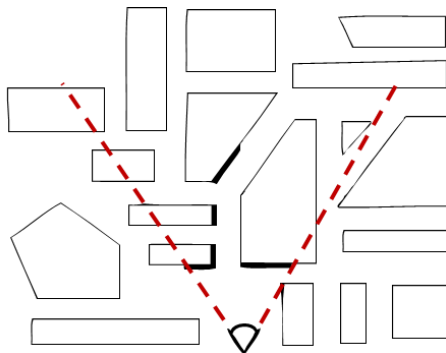
Podsetimo se: protočna obrada kod rasterizacije



Fokus ovog predavanja je na poslednjoj etapi obrade

Problem utvrđivanja vidljivosti

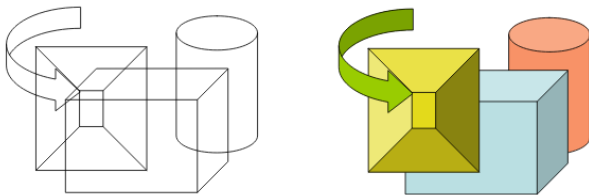
- Renderovati ili ne, pitanje je sad...



Problem utvrđivanja vidljivosti

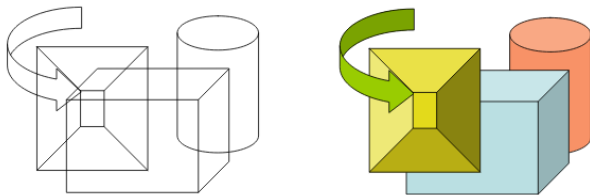
- **Problem utvrđivanja vidljivosti** se bavi odlučivanjem koja se površ na sceni vidi na poziciji nekog piksela
- Fundamentalni problem u računarskoj grafici
- Postoje metode za odbacivanje skrivenih površi koje se izvršavaju pre nego što se procesom rasterizacije poligon konvertuje u fragmente

Klasifikacija algoritama prema Saderlandu, 1973.



- **Algoritmi prostora slika** (image-precision algoritmi)
 - rade nad projekcijama objekata u koordinatama ekrana
 - **za svaki piksel** određuje se koji od objekata je vidljiv, tj. najbliži kameri i piksel se boji odgovarajućom bojom

Klasifikacija algoritama prema Saderlandu, 1973.



- **Algoritmi prostora slika** (image-precision algoritmi)
 - rade nad projekcijama objekata u koordinatama ekrana
 - **za svaki piksel** određuje se koji od objekata je vidljiv, tj. najbliži kameri i piksel se boji odgovarajućom bojom
- **Algoritmi prostora objekta** (object-precision algoritmi)
 - rade direktno nad objektima u svetskom koordinatnom sistemu
 - **za svaki objekat** određuje se deo objekta koji je vidljiv
 - objekti se međusobno porede i eliminišu se kompletni objekti ili delovi objekta koji nisu vidljivi

Vremenska složenost različitih klasa algoritama

- Algoritmi prostora slika
 - složenost = $O(\text{broj_piksela} \cdot \text{broj_objekata})$
 - npr. za $1286 \cdot 1024$ piksela i 1 milion objekata je $O(10^{12})$

Vremenska složenost različitih klasa algoritama

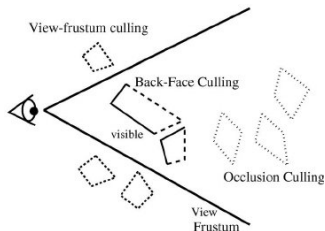
- Algoritmi prostora slika
 - složenost = $O(\text{broj_piksela} \cdot \text{broj_objekata})$
 - npr. za $1286 \cdot 1024$ piksela i 1 milion objekata je $O(10^{12})$
- Algoritmi prostora objekta
 - rezolucija uređaja za prikaz nije relevantna jer se izračunavanja izvode na nivou objekata
 - u najgorem slučaju potrebno je porediti svaki objekat sa svakim
 - složenost = $O(\text{broj_objekata}^2)$
 - npr. za 1 milion objekata je $O(10^{12})$

Klasifikacija algoritama za utvrđivanje vidljivosti

- **Egzaktni algoritmi** – garantuju ispravno utvrđivanje vidljivosti
 - z-bafer algoritam
 - rej kasting algoritam

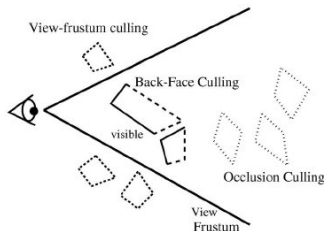
Klasifikacija algoritama za utvrđivanje vidljivosti

- **Egzaktni algoritmi** – garantuju ispravno utvrđivanje vidljivosti
 - z-bafer algoritam
 - rej kasting algoritam
- **Konzervativni algoritmi** – odbacuju deo scene koji sigurno nije vidljiv
 - odbacivanje u odnosu na zapreminu pogleda (view-frustum culling)
 - odbacivanje zadnje strane (naličja) objekta (back-face culling)
 - blokirajuće odbacivanje (occlusion culling)



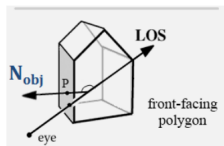
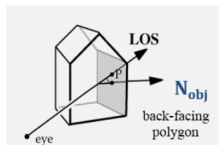
Klasifikacija algoritama za utvrđivanje vidljivosti

- **Egzaktni algoritmi** – garantuju ispravno utvrđivanje vidljivosti
 - z-bafer algoritam
 - rej kasting algoritam
- **Konzervativni algoritmi** – odbacuju deo scene koji sigurno nije vidljiv
 - odbacivanje u odnosu na zapreminu pogleda (view-frustum culling)
 - odbacivanje zadnje strane (naličja) objekta (back-face culling)
 - blokirajuće odbacivanje (occlusion culling)



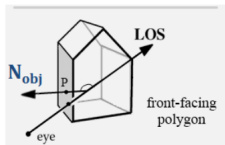
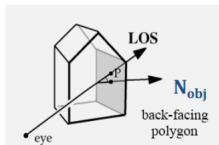
- Često se vrši kombinovanje ovih pristupa

Odbacivanje zadnje strane objekta



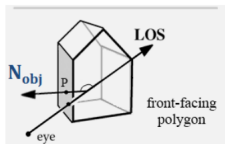
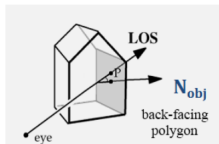
- **Zadnja strana** objekta nije vidljiva iz pozicije kamere

Odbacivanje zadnje strane objekta



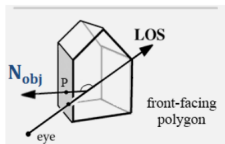
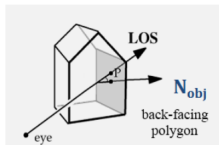
- Zadnja strana objekta nije vidljiva iz pozicije kamere
- Gotovo polovina geometrije scene se može odbaciti
- Neka je posmatrač van objekta i neka gleda u smeru vektora $\overrightarrow{LOS}$
- Kako utvrditi koji poligoni čine zadnju stranu objekta?

Odbacivanje zadnje strane objekta



- Neka je P proizvoljna tačka poligona za koji ispitujemo da li je sa zadnje strane, a $\overrightarrow{N_{obj}}$ vektor normale poligona P

Odbacivanje zadnje strane objekta

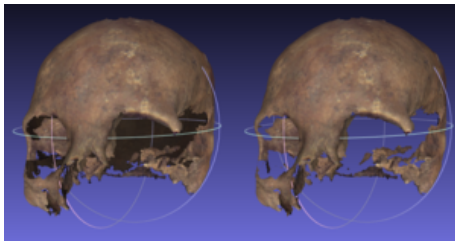


- Neka je P proizvoljna tačka poligona za koji ispitujemo da li je sa zadnje strane, a $\vec{N}_{obj}$ vektor normale poligona P

- Poligon P je:

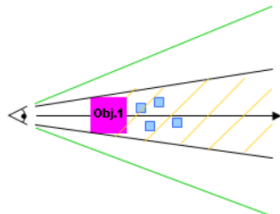
- sa prednje strane u odnosu na posmatrača ako je $\vec{LOS} \cdot \vec{N}_{obj} < 0$
- sa zadnje strane u odnosu na posmatrača ako je $\vec{LOS} \cdot \vec{N}_{obj} > 0$
- na konturi koja razdvaja prednju i zadnju stranu ako je $\vec{LOS} \cdot \vec{N}_{obj} = 0$

Odbacivanje zadnje strane objekta



- Trougao sa zadnje strane u odnosu na posmatrača nije vidljiv samo ako se nešto neprozirno nalazi ispred njega
- Za ovo postoje garancije samo u slučaju **zatvorenih neprozirnih** tela
- Kod objekata koji sadrže “rupe” mogu se videti i poligoni sa zadnje strane u odnosu na posmatrača

Blokirajuće odbacivanje



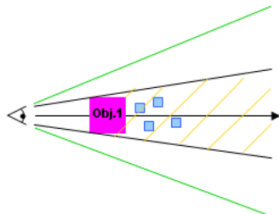
— User's field of view



Area where objects hidden by **Object 1** are not displayed

- Cilj je efikasno identifikovati objekte **zaklonjene drugim objektima** kako bi se izbeglo njihovo razmatranje

Blokirajuće odbacivanje



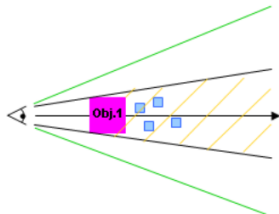
— User's field of view



Area where objects hidden by **Object 1** are not displayed

- Cilj je efikasno identifikovati objekte **zaklonjene drugim objektima** kako bi se izbeglo njihovo razmatranje
- Scena se može sagledati hijerarhijski

Blokirajuće odbacivanje



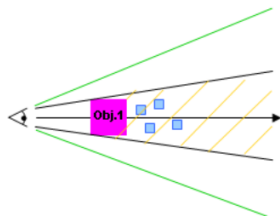
— User's field of view



Area where objects hidden by **Object 1** are not displayed

- Cilj je efikasno identifikovati objekte **zaklonjene drugim objektima** kako bi se izbeglo njihovo razmatranje
- Scena se može sagledati hijerarhijski
- Scena se može particionisati na disjunktne potprostore

Blokirajuće odbacivanje



— User's field of view

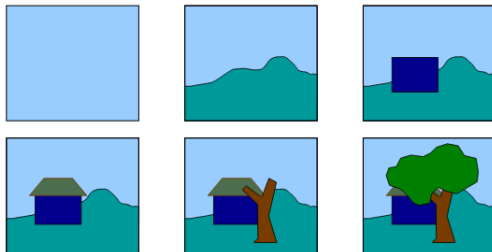


Area where objects hidden by **Object 1** are not displayed

- Cilj je efikasno identifikovati objekte **zaklonjene drugim objektima** kako bi se izbeglo njihovo razmatranje
- Scena se može sagledati hijerarhijski
- Scena se može particionisati na disjunktne potprostore
- Najčešće se koriste namenske prostorne strukture podataka

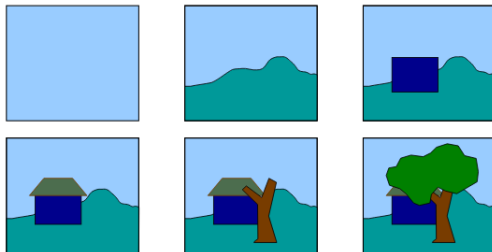
Algoritmi sa listama prioriteta

- **Algoritmi sa listama prioriteta** implicitno rešavaju problem vidljivosti tako što renderuju elemente scene u redosledu od udaljenijih ka bližim
- Dalji objekti imaju viši prioritet i kao takvi se renderuju prvi, a biće naknadno zaklonjeni objektima koji se kasnije renderuju
- Primer: **slikarev algoritam**



Algoritmi sa listama prioriteta

- **Algoritmi sa listama prioriteta** implicitno rešavaju problem vidljivosti tako što renderuju elemente scene u redosledu od udaljenijih ka bližim
- Dalji objekti imaju viši prioritet i kao takvi se renderuju prvi, a biće naknadno zaklonjeni objektima koji se kasnije renderuju
- Primer: **slikarev algoritam**



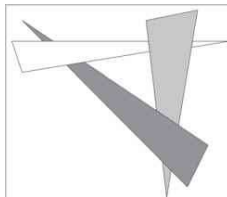
- Korišćeni su u prvom simulatoru letenja koji je računarski generisao slike u realnom vremenu (General Electric, 1967.)
- Danas se retko koriste jer su razvijene bolje alternative

Slikarev algoritam

- Problemi zaklanjanja i utvrđivanja vidljivosti se rešavaju tako što se boje daljih tačaka pregaze bojama bližih tačaka
- Ovu ideju možemo primeniti na:
 - pojedinačne piksele – neefikasno, ali tačno

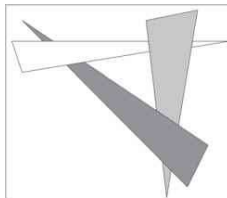
Slikarev algoritam

- Problemi zaklanjanja i utvrđivanja vidljivosti se rešavaju tako što se boje daljih tačaka pregaze bojama bližih tačaka
- Ovu ideju možemo primeniti na:
 - pojedinačne piksele – neefikasno, ali tačno
 - kompletne primitive, npr. trouglove – efikasnije, ali postoje situacije kada ovo nije moguće uraditi



Slikarev algoritam

- Problemi zaklanjanja i utvrđivanja vidljivosti se rešavaju tako što se boje daljih tačaka pregaze bojama bližih tačaka
- Ovu ideju možemo primeniti na:
 - pojedinačne piksele – neefikasno, ali tačno
 - kompletne primitive, npr. trouglove – efikasnije, ali postoje situacije kada ovo nije moguće uraditi



- Može se dozvoliti podela primitiva tamo gde se njihove projekcije seku

Algoritam sortiranja dubine

- **Algoritam sortiranja dubine** predstavlja modifikaciju slikarevog algoritma koja garantuje ispravno renderovanje
- Algoritam se sastoji od četiri koraka:

Algoritam sortiranja dubine

- **Algoritam sortiranja dubine** predstavlja modifikaciju slikarevog algoritma koja garantuje ispravno renderovanje
- Algoritam se sastoji od četiri koraka:
 - 1 svakom poligonu treba dodeliti ključ sortiranja koji je jednak z-vrednosti temena koje je najdalje od oblasti za prikaz

Algoritam sortiranja dubine

- **Algoritam sortiranja dubine** predstavlja modifikaciju slikarevog algoritma koja garantuje ispravno renderovanje
- Algoritam se sastoji od četiri koraka:
 - 1 svakom poligonu treba dodeliti ključ sortiranja koji je jednak z-vrednosti temena koje je najdalje od oblasti za prikaz
 - 2 sortirati poligone od najudaljenijeg do najbližeg prema vrednosti ključa

Algoritam sortiranja dubine

- **Algoritam sortiranja dubine** predstavlja modifikaciju slikarevog algoritma koja garantuje ispravno renderovanje
- Algoritam se sastoji od četiri koraka:
 - 1 svakom poligonu treba dodeliti ključ sortiranja koji je jednak z-vrednosti temena koje je najdalje od oblasti za prikaz
 - 2 sortirati poligone od najudaljenijeg do najbližeg prema vrednosti ključa
 - 3 otkriti slučajeve kada dva poligona imaju dvosmisleno uređenje – ovakve poligone deliti sve dok delovi nemaju eksplicitno uređenje i postaviti ih na pravo mesto u sortiranoj listi

Algoritam sortiranja dubine

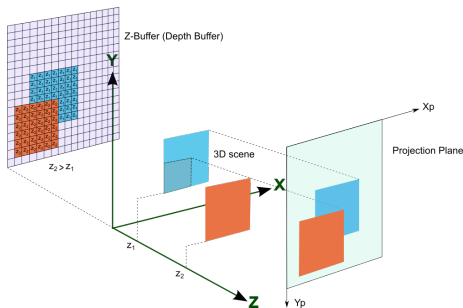
- **Algoritam sortiranja dubine** predstavlja modifikaciju slikarevog algoritma koja garantuje ispravno renderovanje
- Algoritam se sastoji od četiri koraka:
 - 1 svakom poligonu treba dodeliti ključ sortiranja koji je jednak z-vrednosti temena koje je najdalje od oblasti za prikaz
 - 2 sortirati poligone od najudaljenijeg do najbližeg prema vrednosti ključa
 - 3 otkriti slučajeve kada dva poligona imaju dvosmisleno uređenje – ovakve poligone deliti sve dok delovi nemaju eksplicitno uređenje i postaviti ih na pravo mesto u sortiranoj listi
 - 4 renderovati poligone u redosledu prioriteta, od najudaljenijeg do najbližeg

Bafer dubine

- Protočna obrada za problem rasterizacije obrađuje **trougao po trougao**
- Na koji način uporediti više trouglova i saznati koji treba da se vidi u svakoj tački uzorkovanja?

Bafer dubine

- Protočna obrada za problem rasterizacije obrađuje **trougao po trougao**
- Na koji način uporediti više trouglova i saznati koji treba da se vidi u svakoj tački uzorkovanja?
- Rešenje je održavanje bafera dubine
- **Bafer dubine** (ili **z-bafer**) za svaki piksel sadrži po jedan skalar koji daje neku meru rastojanja kamere od površi kojom je obojen piksel

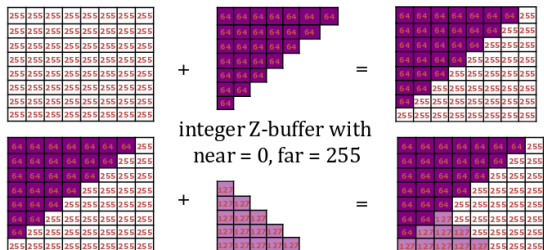


z-bafer algoritam

- Katmul, 1974.
- Sve vrednosti u kolor baferu se inicijalizuju na boju pozadine, a sve vrednosti u z-baferu na z-koordinatu zadnje ravni odsecanja
- Maksimalna vrednost z-bafera je z-koordinata prednje ravni odsecanja
- Nekada se vrednosti u z-baferu čuvaju i kao celobrojne vrednosti

z-bafer algoritam

- Na sve trouglove koji predstavljaju scenu primenjuje se algoritam rasterizacije, u proizvoljnom poretku
- Prilikom rasterizacije trougla na poziciji (x, y) , ako ta tačka nije udaljenija od kamere u odnosu na tačku čija se z-koordinata trenutno nalazi u z-baferu na poziciji tog piksela, onda z koordinata te tačke zamenjuje postojeću vrednost u z-baferu, a njena boja se smešta u kolor bafer na poziciju (x, y)



Pseudokod algoritma

```
Procedure zBuffer
```

```
var
```

```
    pz: integer;
```

```
begin
```

```
    for y := 0 to YMAX do
```

```
        for x := 0 to XMAX do
```

```
            begin
```

```
                WritePixel(x,y,background_value);
```

```
                WriteZ(x,y,1)
```

```
            end
```

```
za svaki poligon uradi sledece
```

```
za svaki fragment u projekciji poligona uradi sledece
```

```
begin
```

```
    pz := z-vrednost poligona u tacki sa koordinatama (x,y);
```

```
    if pz <= ReadZ(x,y) then
```

```
        begin { ova tacka nije dalja }
```

```
            WriteZ(x,y,pz);
```

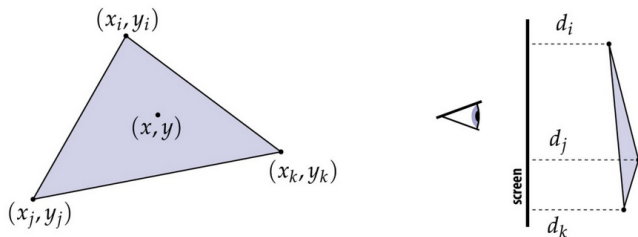
```
            WritePixel(x,y,boja_poligona_u_tacki(x,y))
```

```
        end
```

```
    end
```

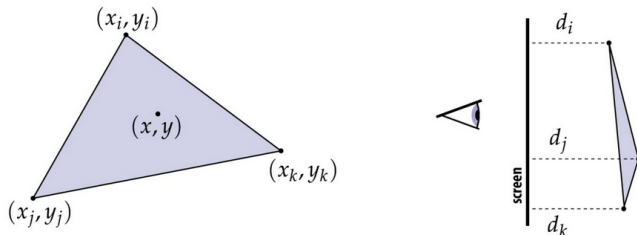
```
end.
```

Uzorkovanje dubine



- Pretpostavimo da je trougao zadat:
 - projektovanim 2D koordinatama (x_i, y_i) temena
 - dubinom d_i svakog temena (rastojanjem do posmatrača)
- Kako izračunati dubinu d u proizvoljnoj tački trougla (x, y) u kojoj vršimo uzorkovanje?

Uzorkovanje dubine



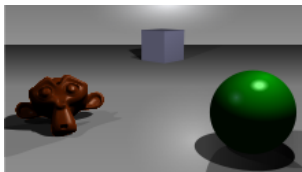
- Recipročna vrednost dubine se linearno menja duž trougla
- Vršimo interpolaciju recipročnih vrednosti dubine u temenima korišćenjem baricentričnih koordinata (detaljnije naredne nedelje)

Ilustracija sadržaja z-bafera

- Za ilustraciju vrednosti u z-baferu možemo koristiti nijanse sive
 - vrednosti bliske beloj boji odgovaraju udaljenim objektima
 - vrednosti bliske crnoj boji odgovaraju bližim objektima

Ilustracija sadržaja z-bafera

- Za ilustraciju vrednosti u z-baferu možemo koristiti nijanse sive
 - vrednosti bliske beloj boji odgovaraju udaljenim objektima
 - vrednosti bliske crnoj boji odgovaraju bližim objektima

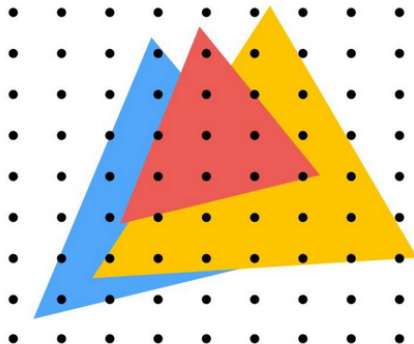


A simple three-dimensional scene



Z-buffer representation

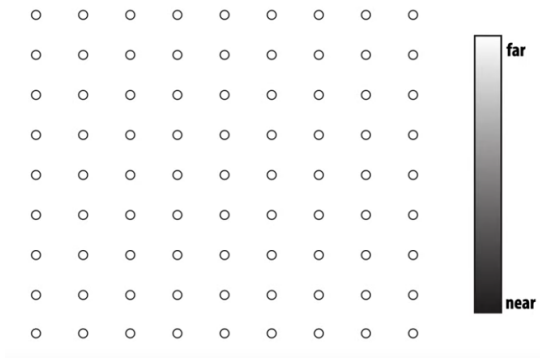
Primer: renderovanje tri neprozirna trougla



Preuzeto sa: <http://15462.courses.cs.cmu.edu/spring2021/lecture/alpha>

Primer: renderovanje tri neprozirna trougla

Vrednosti u z-baferu inicijalizujemo na belu



Primer: renderovanje tri neprozirna trougla

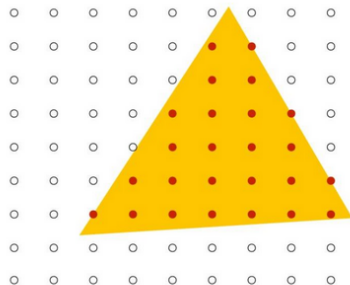
Processing yellow triangle:
depth = 0.5



Color buffer contents

near  far

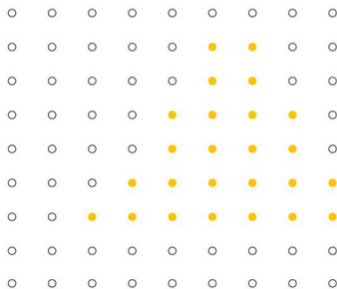
● — sample passed depth test



Depth buffer contents

Primer: renderovanje tri neprozirna trougla

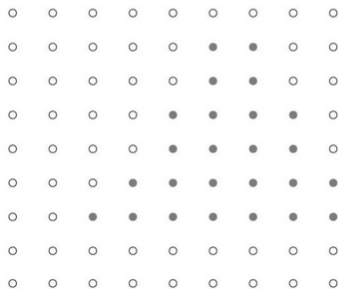
After processing yellow triangle:



Color buffer contents

near  far

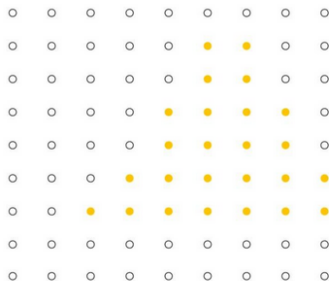
● — sample passed depth test



Depth buffer contents

Primer: renderovanje tri neprozirna trougla

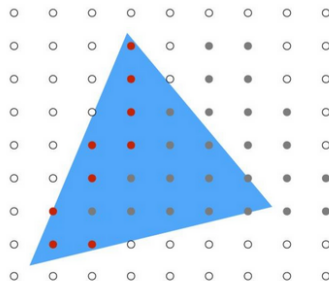
Processing blue triangle:
depth = 0.75



Color buffer contents

near  far

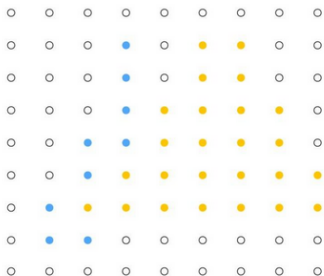
● — sample passed depth test



Depth buffer contents

Primer: renderovanje tri neprozirna trougla

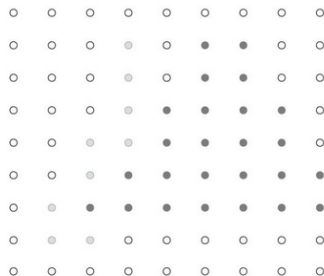
After processing blue triangle:



Color buffer contents

near  far

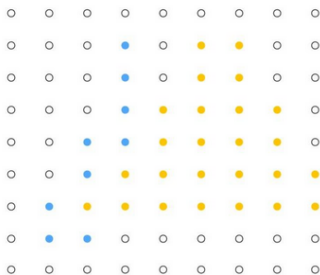
● — sample passed depth test



Depth buffer contents

Primer: renderovanje tri neprozirna trougla

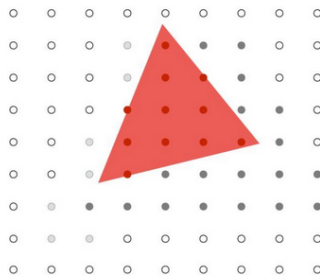
Processing red triangle:
depth = 0.25



Color buffer contents

near far

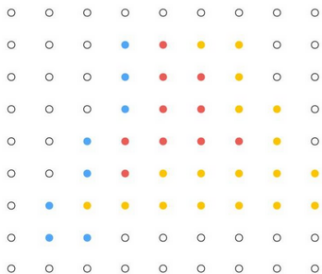
● — sample passed depth test



Depth buffer contents

Primer: renderovanje tri neprozirna trougla

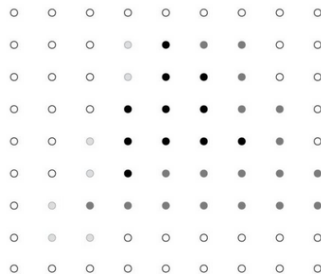
After processing red triangle:



Color buffer contents

near far

● — sample passed depth test



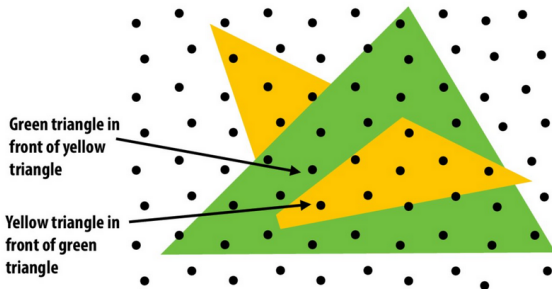
Depth buffer contents

Dubina + preseci primitiva

- Dosadašnje razmatranje je pretpostavljalo da su trouglovi u celosti jedan ispred drugog
- Da li z-bafer algoritam podržava površi koje se presecaju?

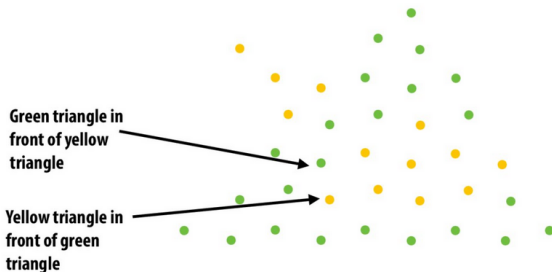
Dubina + preseci primitiva

- Dosadašnje razmatranje je pretpostavljalo da su trouglovi u celosti jedan ispred drugog
- Da li z-bafer algoritam podržava površi koje se presecaju? Da
- Test da li jedna primitiva zaklanja drugu se zasniva na dubini trougla u tački u kojoj vršimo uzorkovanje
- Dubina trougla može biti različita u različitim tačkama trougla



Dubina + preseci primitiva

- Dosadašnje razmatranje je pretpostavljalo da su trouglovi u celosti jedan ispred drugog
- Da li z-bafer algoritam podržava površi koje se presecaju? Da
- Test da li jedna primitiva zaklanja drugu se zasniva na dubini trougla u tački u kojoj vršimo uzorkovanje
- Dubina trougla može biti različita u različitim tačkama trougla

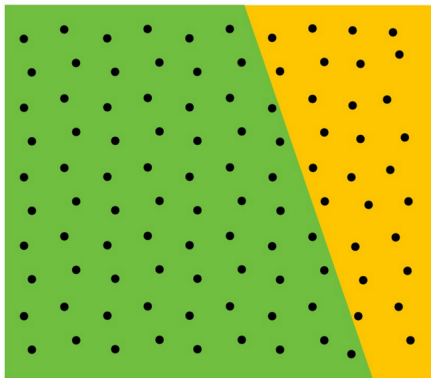


Dubina + nadsemplovanje

- Da li z-bafer podržava nadsemplovanje?

Dubina + nadsemplovanje

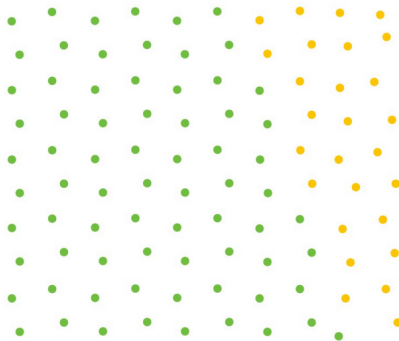
- Da li z-bafer podržava nadsemplovanje? Da
- Moguće je u z-baferu čuvati dubine većeg broja uzoraka po pikselu



(Here: green triangle occludes yellow triangle)

Dubina + nadsemplovanje

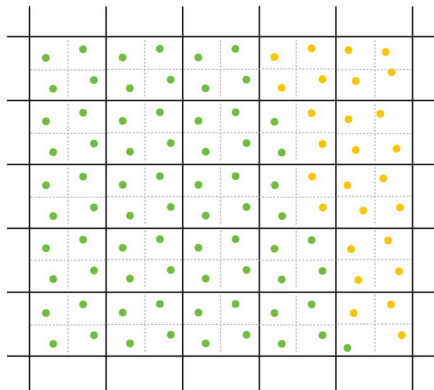
- Da li z-bafer podržava nadsemplovanje? Da
- Moguće je u z-baferu čuvati dubine većeg broja uzoraka po pikselu



Color of super samples after rasterizing w/ depth buffer

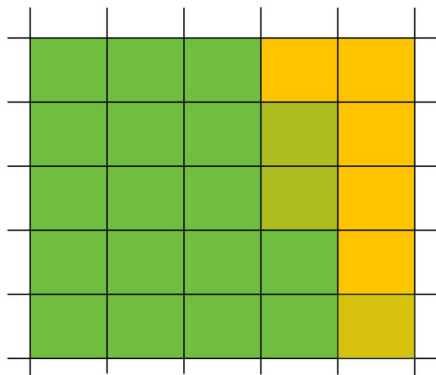
Sadržaj kolor bafera

- Kolor bafer može da sadrži jedan ili veći broj uzoraka boje po pikselu
- Primer kada imamo 4 uzorka po pikselu



Krajnji rezultat

- Računamo prosek vrednosti uzoraka po pikselu
- Boja se postepeno i glatko menja pri prelasku iz levog poligona u desni



Rezime z-bafer algoritma

- Pamtimo jednu vrednost dubine po uzorku, a ne po pikselu
- Podržava nadsemplovanje po cenu veće vremenske i memorijske složenosti algoritma
- z-bafer algoritam koristi konstantan prostor po uzorku, ne zavisi od broja primitiva koje se preklapaju
- Nije potrebno sortiranje objekata pre primene algoritma
- z-bafer algoritam nije ograničen na trouglove: jedino je potrebno omogućiti računanje dubine površi u tački u kojoj vršimo uzorkovanje
- z-bafer algoritam može biti implementiran u hardveru

Slabosti z-bafer algoritma

- Vrednosti u z-baferu su predstavljene konačnim brojem bitova (npr. 8 bitova daje vrednosti iz opsega $\{0, 255\}$)
- Ako dva različita fragmenta imaju bliske vrednosti dubine, oni mogu odgovarati istoj vrednosti u z-baferu i nećemo znati koji je ispred kog
- Da li se ovo često dešava u praksi?

Slabosti z-bafer algoritma

- Vrednosti u z-baferu su predstavljene konačnim brojem bitova (npr. 8 bitova daje vrednosti iz opsega $\{0, 255\}$)
- Ako dva različita fragmenta imaju bliske vrednosti dubine, oni mogu odgovarati istoj vrednosti u z-baferu i nećemo znati koji je ispred kog
- Da li se ovo često dešava u praksi? Da
- Posledica je delovanja matrice transformacije standardne perspektivne u standardnu paralelnu zapreminu pogleda

$$M_{pp} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/(1+c) & -c/(1+c) \\ 0 & 0 & -1 & 0 \end{bmatrix}, \text{ gde je } c = -n/f$$

Perspektivna transformacija uzrokuje z-kompresiju

- Razmotrimo kako matrica M_{pp} deluje na tačku sa koordinatama (x, y, z) :

$$M_{pp} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/(1+c) & -c/(1+c) \\ 0 & 0 & -1 & 0 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ \frac{z-c}{1+c} \\ -z \end{bmatrix}$$

Perspektivna transformacija uzrokuje z-kompresiju

- Razmotrimo kako matrica M_{pp} deluje na tačku sa koordinatama (x, y, z) :

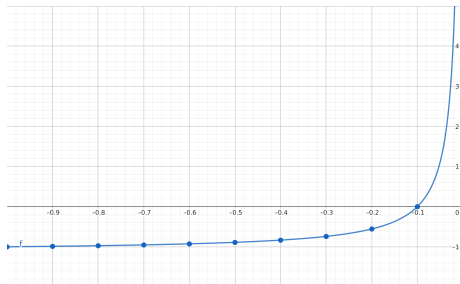
$$M_{pp} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/(1+c) & -c/(1+c) \\ 0 & 0 & -1 & 0 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ \frac{z-c}{1+c} \\ -z \end{bmatrix}$$

- Nakon homogenizacije dobijamo $\begin{bmatrix} x/-z \\ y/-z \\ \frac{c-z}{z+zc} \\ 1 \end{bmatrix}$

- Sa povećanjem z koordinate veći je stepen perspektivnog skraćanja po x i y
- Označimo novu vrednost dubine (tj. z -koordinate) $\frac{c-z}{z+zc}$ sa z'

Perspektivna transformacija uzrokuje z-kompresiju

- Razmotrimo odnos nove vrednosti z-koordinate u odnosu na polaznu
- Fiksirajmo vrednosti parametara n i f , npr. $n = 0.1$ i $f = 1$
- Samim tim fiksirana je i vrednost c ($c = -0.1$)



- Razmatramo z-vrednosti između zadnje i prednje ravni odsecanja $z \in [-1, -0.1]$
- Vrednosti dubine se nagomilavaju oko vrednosti $z = -1$ u standardnoj paralelnoj zapremini pogleda - ovaj efekat naziva se **z-kompresija**

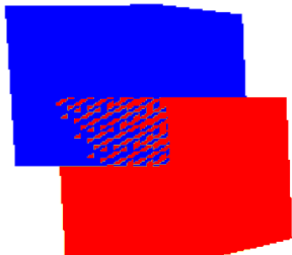
z-kompresija viđena algebarski

- Transformacija standardne perspektivne u standardnu paralelnu zapreminu pogleda uzrokuje **nelinearnu kompresiju ka zadnjoj ravni odsecanja** $z = -1$
- Što je prednja ravan odsecanja bliža ravni $z = 0$, a zadnja ravan odsecanja udaljenija, vrednost $c = -n/f$ je bliskija vrednosti 0, a efekat z-kompresije je izraženiji

$$z' = \frac{c - z}{z + zc} \approx -\frac{z}{z} = -1$$

Vizualni efekat z-kompresije: z-konflikt

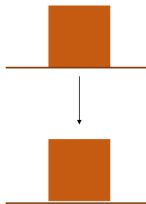
- **z-konflikt** se javlja kada dve primitive imaju slične vrednosti u z-baferu
 - komplanarni poligoni
 - jedan se na proizvoljni način bira da bude iznad drugog, ali vrednost z-koordinate varira duž poligona i javlja se ovaj efekat (<https://www.youtube.com/watch?v=CjckWVwd2ek>)



- Posebno kritično kada se kamera kreće

Kako se boriti sa z-konfliktom?

- Izbegavati da se objekti postavljaju preblizu jedan drugome tako da se neki njihovi trouglovi preklapaju
- Napraviti mali pomak koji je za korisnika jedva primetan



- Povećati **preciznost** bafera dubine (npr. koristiti 32-bitni z-bafer koji čuva realne brojeve) po cenu povećanja memorijskih zahteva
- Povećati vrednost razlomka n/f
 - zadnju ravan odsecanja primaknuti posmatraču
 - prednju ravan odsecanja odmaknuti od posmatrača

Transparentni poligoni

- Primetimo da stvari postaju komplikovanije ako su trouglovi transparentni

